9. Iterating Through Collections — Quick Notes

Ways to Traverse Collections

Method	Notes
Iterator	Can remove items while traversing
for-each loop	Simple traversal
forEach() method	Uses Lambda
Stream API	Most recommended (Java 8+)

1. Iterator Traversal

Purpose

- Traverse collection
- Remove items safely

Example:

```
Iterator<Integer> it = list.iterator();

while(it.hasNext()) {
   System.out.println(it.next());
}
```

ListIterator

Available for: ArrayList, LinkedList, Vector

Extra feature: Backward traversal

2. for-each Loop

Simple Traversal

Example:

```
List<Integer> list = Arrays.asList(10,20,30);

for(Integer x : list)
   System.out.println(x);
```

Output:

10
20
30

for-each Syntax

for(datatype var : collection)

Example: for(Integer x : list)

Important Point

Works for:

- ArrayList
- LinkedList
- HashSet
- TreeSet
- ArrayDeque

Because all implement: Iterable

3. forEach() Method

Present in: Iterable Interface

Uses:

- Lambda Expressions
- Consumer Interface

Basic Example

list.forEach(x -> System.out.println(x));
Output:
10
20
30

Method Reference

Instead of: x -> System.out.println(x)

Can write: System.out::println

Example: list.forEach(System.out::println);

Lambda Syntax

parameter -> logic

Example: x -> x * 2

4. Stream Traversal (Java 8+)
Most Recommended

Supports:

- Filtering
- Mapping
- Aggregation
- Pipeline processing

Basic Stream Example

```
list.stream()
   .forEach(x -> System.out.print(x + " "));
```

Output: 10 20 30

Stream Filtering Example

```
list.stream()
   .filter(x -> x > 10)
   .filter(x -> x % 2 == 0)
   .forEach(System.out::println);
```

Output:

20
40

Stream Pipeline
Intermediate Operations

- filter()
- map()

Terminal Operations

- forEach()
- count()
- findFirst()

Important Point

Streams use: Lazy Evaluation

Meaning: operations execute only when terminal operation runs

Stream Advantages

Feature	Benefit
Pipeline processing	Cleaner code
Lazy evaluation	Better performance
Parallel streams	Faster execution
Functional style	Concise code